31927 - Applications Development with .NET
32998 - .NET Applications Development
SPRING 2026
ASSIGNMENT-2 COVER SHEET

PROJECT TITLE	FlatPlate
LEAD STUDENT NAME	Jonathan Immanuel
LEAD STUDENT ID	14487692
GROUP MEMBER NAME(S) AND STUDENT ID(S)	Yilin Wang 14605796 Linhai Shi 26502310
SUBMISSION DATE	17 October 2026

AI USE RESTRICTIONS FOR PROGRAMMING WORK

Do not use AI to generate a substantial or complete solution to the programming assignment, or to bypass the learning outcomes of the assessment.

- AI must not be used to invent or misrepresent code execution, test results, screenshots, datasets, performance measurements, external libraries/APIs used, contributions of group members, or any other project detail.
- Permitted support may include concept explanations, syntax help, debugging suggestions, code review, test-case ideas, partial code generation, documentation/comments, and limited code examples where consistent with the assessment instructions.
- You must understand, review, run and test any AI-assisted code yourself, verify that it meets the specification, and be able to explain every submitted component and design decision.
- All AI use must be disclosed. AI output is not an authoritative source and does not replace reading subject materials, documentation, or instructions from teaching staff.

DECLARATION OF ORIGINALITY AND RESPONSIBILITY

· I confirm that the submitted work is the work of the named student(s), except where collaboration, source code, libraries, data, or AI assistance has been appropriately acknowledged. I understand the submitted program and can explain its implementation, testing and results.

Lead student signature: Jonathan Immanuel Date: 17 October 2026

Group member signature(s):

Linhai Shi Date: 17 October 2026
Yilin Wang

AI USE STATEMENT

Attach this completed statement as an appendix to your submission. Disclose all AI-enabled tools used, including uses that did not produce code retained in the final submission.

Student name(s) and ID(s)	Jonathan Immanuel
Subject code	· 31927 ☐ 32998
Project / assessment title	Assignment 2

· I did use any AI-enabled tool for this assessment. (If selected, proceed to the declaration.)

1. AI tools and model names

List every AI-enabled tool used (for example ChatGPT, GitHub Copilot, Microsoft Copilot, Gemini, Claude and Codex, Grammarly or other coding assistants). Record the model name shown in the tool, or "not shown".

AI tool or product	Model name (as shown)	Date(s) used
Claude	Sonnet	Sep 30 and Oct 2026
Codex	GPT-6.1 sol and GPT-5	09/Oct/2026

2. What the AI tools were used for (tick all that apply)

· Explained programming concepts or APIs	· Suggested algorithms, approaches or pseudocode
☐ Helped with syntax or small code examples	☐ Generated or completed code
· Debugged errors or explained error messages	· Reviewed/refactored code I had written
· Suggested unit tests, edge cases or test data	☐ Generated documentation, comments or README text
☐ Helped with data analysis or visualisation code	· Assisted with version-control / build / tooling commands
☐ Proofread or revised report text	☐ Other:

3. Representative examples of AI use

Give 2–3 examples. Summarise what you asked the tool to do and explain what you accepted, changed, tested or rejected. Do not paste full chat logs unless requested.

Tool / model	Prompt or task summary	How I used, changed or rejected the output	Where it affected the submission
Claude Sonnet and Codex GPT-5	Give me project ideas based on the Assignment 2 pdf i attached, give me a realistic idea that is viable for the timeline given.	We checked the suggestions against the assignment requirements. And when we liked it, we prompted AI to help us plan and make what models to make.	Project planning, shared models
Codex GPT-6.1 sol	Could you please help search for 60 food ingredients on the website from Woolworths. It has to include price, name, weight and description. After finishing searching, please put all data in a Markdown file, I can check the information easily.	Use the computer controller to search lots of data about food ingredients, including milk, meat, fruit and so on.	Data collection, but changed to FreshCrawl
Codex GPT-5.6 Sol	Help me set up azure using the azure for students package. Show me clear concise steps on what to deploy until finished.	Used the steps to configure Azure, from azure app function, database,etc., until the app is deployed.	Homepage, authentication

4. Verification and validation

· Ran and tested all submitted code and reviewed outputs, errors and edge cases	· Checked AI-suggested APIs/libraries against official documentation or course materials
· Confirmed that reported outputs, screenshots and performance results came from my/our actual program execution	☐ Reviewed AI-assisted code for correctness, security, originality, licensing/attribution and assessment requirements
· I/we can explain every submitted code component and justify the implementation and testing decisions	· No confidential, personal or unauthorised third-party material was uploaded to an AI tool

Briefly describe one specific check you performed and any change made as a result:

I ran the final solution build and NUnit test suite after integrating the team branches. This identified integration issues in the application shell and navigation, which were corrected before rerunning the build and tests.

Declaration

I/we confirm that this statement accurately describes all AI-enabled tools used for this assessment and the extent to which their outputs influenced or appear in the submission. I/we followed the assessment instructions, independently reviewed and tested the submitted work, and take responsibility for the final code, report and results. I/we understand that prompts, outputs, commit history or other supporting evidence may be requested for an academic integrity review.

Lead student signature: Jonathan Immanuel Date: 17 October 2026

Group member signature(s): Linhai Shi and Yilin Wang Date: 17 October 2026

Table of Content

AI USE STATEMENT 2
1. AI tools and model names 2
2. What the AI tools were used for (tick all that apply) 2
3. Representative examples of AI use 2
4. Verification and validation 3
Declaration 3
Table of Content 4
1 Introduction and Project Summary 5
1.1 Project Overview 5
1.2 Motivation and Objectives 5
1.3 Key Features 5
2 Development Approach 6
2.1 Development Methodology 6
2.2 Programming Techniques 6
2.3 Frameworks Tools and Price Data 7
2.4 Integration Validation and Testing 7
3.1 System Flowchart 9
Figure 1. FlatPlate component, service and data interaction flow 9
4 Role of Team Members 10
4.1 Jonathan Immanuel 10
4.2 Linhai Shi 10
4.3 Yilin Wang 10
4.4 Summary Table 11
5 References 12

1 Introduction and Project Summary

1.1 Project Overview

FlatPlate is a .NET 9 WPF desktop application for planning seven days of meals while monitoring a household grocery budget. Users can maintain recipes, place meals into breakfast, lunch and dinner slots, select a supermarket and set a weekly budget. The application scales recipe quantities by servings, merges repeated ingredients into a single shopping requirement and calculates the number of whole packs that must be purchased. It then compares the planned spend with the selected budget and identifies ingredients whose prices require attention.

The application contains a Home dashboard and five connected functional screens: Recipe Library, Price Book, Weekly Planner, Shopping List and Insights. Account registration, email verification, sign-in and password reset protect access to the workspace. Shared models and services ensure that recipes, prices and planned meals produce consistent results across the screens.

1.2 Motivation and Objectives

Meal planning often requires people to move between recipe websites, notes, supermarket pages and budget calculations. This makes it easy to overlook duplicate ingredients, underestimate pack costs or lose track of items already available at home. FlatPlate was designed to bring these activities into one clear workflow and make the effect of each meal choice visible before shopping begins.

The main objectives were to reduce repeated calculations, support realistic store-specific budgeting and provide clear feedback when data is incomplete or invalid. The team also aimed to demonstrate object-oriented design, persistent data, reusable interfaces, LINQ processing, automated tests and a functional multi-screen WPF interface.

1.3 Key Features

- Recipe management with cuisine, preparation time, servings and measured ingredients.
- A seven-day planner with three meal slots per day, a store selector and adjustable budget.
- Automatic ingredient scaling, unit conversion, merging and whole-pack cost calculation.
- Editable Woolworths and Coles price records with CSV import and export support.
- A grouped shopping list with purchased, already-owned and actual-price tracking.
- Insights for store comparison, cuisine spending and ML.NET missing-price estimation.
- Account registration, verification, sign-in and password recovery through the deployed authentication service.

2 Development Approach

2.1 Development Methodology

FlatPlate was developed using an Agile-inspired iterative approach. The team first agreed on the project scope, shared models, database entities and service interfaces. Establishing these foundations early allowed each member to develop an allocated feature while maintaining compatible data structures and reducing integration conflicts.

Development was organised through GitHub using separate branches for each ownership area. Team members made focused commits, tested their changes locally and merged completed work into the main branch after integration checks. The work progressed from shared models and persistence, to feature screens, then to cross-screen integration, authentication and interface refinement.

This approach was appropriate because FlatPlate contains connected features rather than isolated pages. For example, the Weekly Planner depends on recipe and price data, while the Shopping List, Insights and Home dashboard depend on the saved weekly plan. Incremental integration made these dependencies visible early and reduced the risk of major issues near the submission date.

2.2 Programming Techniques

FlatPlate applies object-oriented programming to model ingredient measurements accurately. "*Ingredient"* is an abstract base class with "*WeightIngredient", "VolumeIngredient"* and "*CountIngredient"* subclasses. Each subtype overrides "*ToBaseAmount()"* to convert supported units into grams, millilitres or individual pieces. "*IngredientMerger"* can therefore process all ingredient types through the common "*Ingredient"* model while applying the correct conversion behaviour.

Encapsulation is used to control how data is accessed and modified. For example, the Price Book view model keeps its editable collection private and exposes a read-only collection for display. Changes are made through commands and validation methods rather than allowing other components to alter records directly.

Interfaces and dependency injection reduce coupling between features. *"IPriceProvider"* defines how current prices are retrieved, while "*DatabasePriceProvider"* supplies the implementation used by the application. *"BudgetCalculator"* receives an "*IPriceProvider"* through its constructor, allowing it to calculate costs without depending directly on a particular database or CSV file. This also allows a future API-based provider to be introduced without changing the budgeting logic.

Generics are used through "Repository<T>" and *"IRepository<T>"* to provide reusable persistence operations for multiple EF Core entities. LINQ supports ingredient processing with methods such as *SelectMany, GroupBy, Sum and OrderBy*. These methods collect recipe ingredients, scale them by servings, group matching items and produce a consolidated shopping requirement.

The application follows the MVVM pattern. View models expose commands and observable properties for the WPF interface, while "*WeeklyPlanService"* raises *"PlanChanged"* after a saved plan change so that dependent screens can refresh their displayed summaries.

2.3 Frameworks Tools and Price Data

FlatPlate is developed in C# using .NET 9. The desktop interface is built with WPF and XAML, which provide the application screens, navigation and data-binding support. Entity Framework Core maps application entities, including recipes, ingredients, price records, planned meals and shopping-list records, to a local SQLite database.

ML.NET is used behind the *"IPriceEstimator"* interface to estimate a missing ingredient price when no suitable price record is available. NUnit provides automated tests for the application's core logic and service boundaries. Visual Studio was used for development, debugging and executing the application, while GitHub was used for source control, feature branches and integration of team members' work.

FlatPlate uses Azure Functions to host the authentication API separately from the WPF desktop application. The WPF client sends registration, sign-in, email-verification and password-reset requests through an HTTP authentication client. The API validates requests, stores authentication records in the deployed Azure SQL Database (sqldb-flatplate-auth) and sends verification or password-reset messages through SMTP email. Separating authentication from the desktop application keeps account security responsibilities independent from meal-planning functions. Sensitive settings, such as connection strings and email credentials, are stored in deployment configuration rather than hard-coded in the application.

The application uses seeded Woolworths and Coles price data derived from a documented subset of the FreshCrawl Australian Supermarket Prices dataset (FreshCrawl, n.d.). The selected data has a snapshot date of 17 September 2026. Seeded data was chosen because the team does not have authorised access to live supermarket APIs. It gives the application reproducible demonstration data, avoids unauthorised scraping and still allows users to edit Price Book entries or import supported CSV files.

If authorised Woolworths API access became available, the system could introduce a "*WoolworthsApiPriceProvider"* implementing "*IPriceProvider"*. It would retrieve approved product and price information, match products to FlatPlate ingredients, normalize package sizes and units, record source and update dates, and refresh the local Price Book. Secure credential storage, API rate-limit handling, retry behaviour and seeded-data fallback would ensure that the planner remains usable if the external service is unavailable.

2.4 Integration Validation and Testing

FlatPlate integrates its features through shared models, repositories and service interfaces. Recipe Library provides recipes and ingredients, Price Book provides store-specific prices, Weekly Planner saves planned meals, Shopping List combines ingredient requirements and Insights compares spending information. The Home dashboard presents a summary of the same saved planning data, preventing screens from calculating totals from separate or outdated information.

Validation occurs before data is stored or imported. Recipe operations require valid names, servings and ingredients. Price operations require valid decimal values, positive package sizes and unique ingredient-store combinations. Ingredient conversion checks that measurement units are compatible. CSV import previews check headers, required values, invalid rows and duplicate records before the

user confirms an import. Authentication checks email format, password rules and verification-code handling. Invalid values display field-level feedback or user-friendly errors and are not persisted.

“AppWriteTransaction” protects bulk import operations by committing all valid changes together or rolling them back if an error occurs. This prevents a partially imported CSV file from leaving the Price Book in an inconsistent state.

Automated NUnit tests cover unit conversion, ingredient merging, budgeting, repository operations, weekly-plan behaviour, import validation, store comparison, authentication boundaries and ML.NET estimation. Manual end-to-end testing was also used to confirm complete workflows, such as adding a recipe, assigning it to a meal slot, reviewing budget results, updating a price and confirming that related screens refresh correctly. The final solution build and full automated test suite should be rerun before submission, with only verified results reported in the final report.

3 System Flowchart and Usage Instructions

3.1 System Flowchart

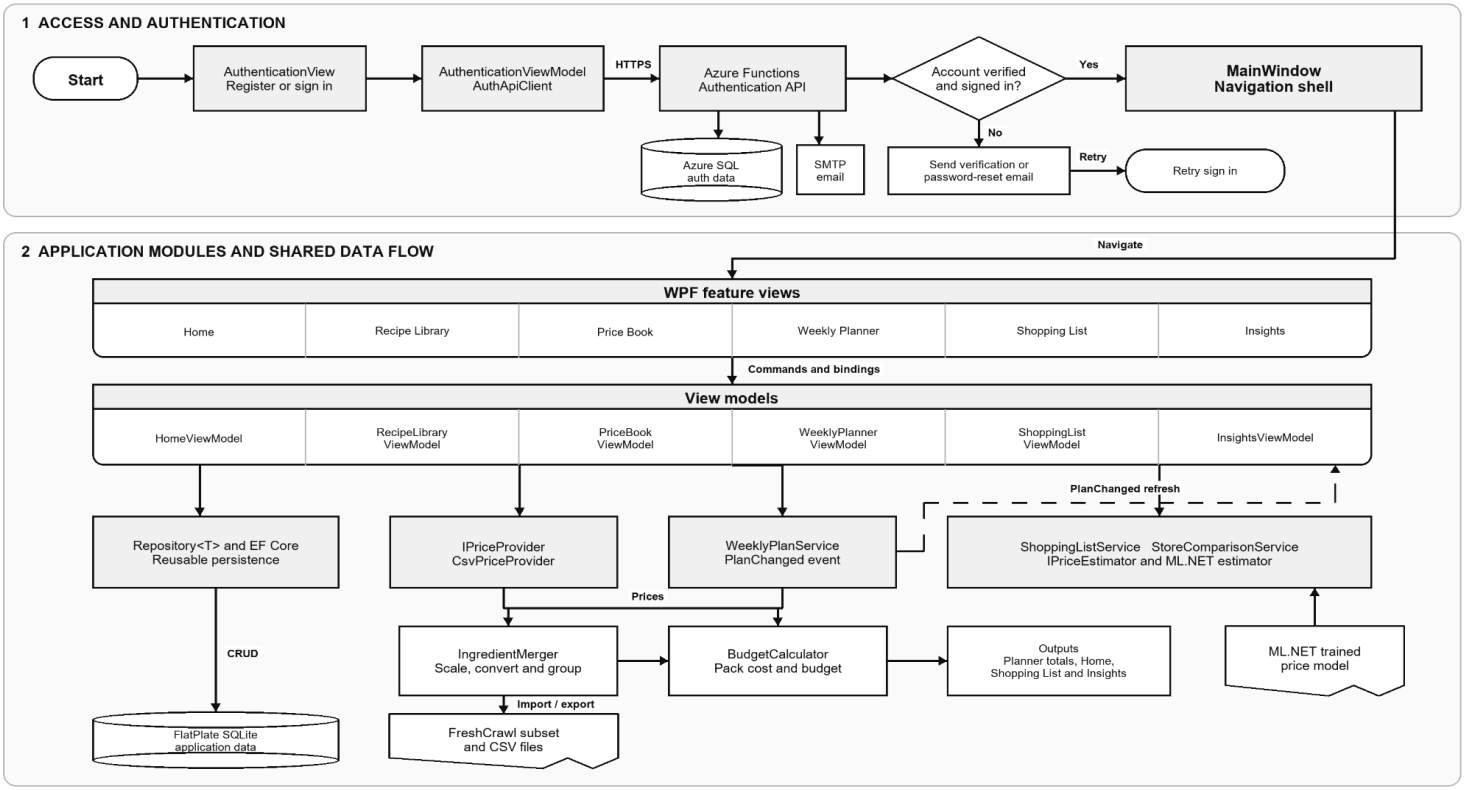


Figure 1. FlatPlate component, service and data interaction flow

The system flowchart shows how FlatPlate's user interface, view models, shared services and data sources interact. After authentication, MainWindow navigates between the application screens. View models use shared services to manage recipes, prices, weekly plans, shopping records and insights. EF Core repositories persist planning data in SQLite, while the Azure Functions authentication API stores account records in Azure SQL Database. Price providers, CSV data and ML.NET support price resolution. When the weekly plan changes, PlanChanged refreshes the dependent Home, Shopping List and Insights calculations.

4 Role of Team Members

The project was divided into 3 ownership areas, with shared responsibility for integration, review and the final report. Each member must review the description below and ensure it matches the work they can demonstrate and explain.

4.1 Jonathan Immanuel

I led the shared core and weekly-planning work. My responsibilities included solution and repository setup, EF Core models and DbContext configuration, the generic Repository<T>, the Ingredient hierarchy, unit conversion, LINQ-based ingredient merging and weekly budget calculation.

I developed the Weekly Planner and Home dashboard, added the Woolworths and Coles seed-price data used by the planner, and developed the core NUnit coverage. I also implemented the Azure Functions authentication API and WPF client integration for account registration, sign-in, email verification and password reset. This deployment stores authentication data in Azure SQL Database and uses SMTP for verification and password-reset email.

As lead student, I integrated team branches, resolved cross-feature issues and completed final navigation and interface-consistency work. I also verified the main branch by building the solution and running the automated test suite.

4.2 Linhai Shi

I developed the Recipe Library, Price Book and CSV workflows using the team's shared models and EF Core infrastructure.

For the Recipe Library, I implemented recipe searching, cuisine filtering and validated Add/Edit operations with SQLite persistence. Then I prepared eight additional recipes with 69 ingredient associations. The manual import skips matching entries and rejects conflicts, preserving existing user edits.

For the Price Book, I implemented store filtering, validated price editing, CSV import and export. Import previews can identify invalid rows and duplicate ingredient-store pairs. Transactional saving prevents partial imports and accidental overwrites, while exports contain only the currently visible rows.

Furthermore, I added a separate importer for 40 ingredient-store price pairs from FreshCrawl's 17 September 2026 snapshot. It validates source hashes, product identifiers, prices, package quantities, units and dates before explicit user confirmation and revalidates them inside the database transaction. These values are historical references, not real-time prices.

In isolated integration tests, 49 regression scenarios passed, including rollback checks. The historical importer was merged into main through PR #3. I used AI assistance for Git operations and commits, collecting data from Woolworths on Chrome, debugging and identifying potential errors, while reviewing its outputs.

4.3 Yilin Wang

I am primarily responsible for the Shopping List, Insights, and machine learning-related features in the FlatPlate project.

For the shopping list section, the development focuses primarily on the shopping list interface and related features. The system displays the ingredients required for the recipes selected in the “Weekly

Planner” and categorizes them by shopping section. Users can select items they've already purchased, mark ingredients they already have at home, and enter the actual purchase price of each item. This makes it easier to manage their shopping lists.

For the Insights part, I am primarily responsible for analyzing and visualizing purchasing data, including consumption trends across different cuisines, supermarket price comparisons, and food waste statistics. In addition, I implemented the calculation logic for Store Comparison, which compares the estimated total cost of purchasing the same items at different supermarkets.

For the Machine Learning section, I use ML.NET and CSV dataset to build a price estimation model, and connect the prediction functionality to other modules by the `IPriceEstimator` interface. This allows the system to provide estimated results when actual prices are missing.

4.4 Summary Table

Member	Primary ownership	Report contribution
Jonathan	Core logic, planner, seed data, tests, Home and Azure authentication	Overview, motivation, objectives and features, system flowchart, revised section 2
Linhai	Recipe Library, Price Book and CSV workflows	Programming techniques, system flowchart
Yilin	Shopping List, Insights, comparison and ML estimation	Methodology, tools, integration and testing

5 References

1. FreshCrawl. Australian Supermarket Prices dataset. Hugging Face. https://huggingface.co/datasets/FreshCrawl/australian-supermarket-prices. Accessed 11 October 2026.